

SKULD LENGUAJE Y COMPILADOR

Alan Gael Gallardo Jiménez

ID: 351914

Carlos Enrique Blanco Ortiz

ID: 349388

8 “A”

Ingeniería en sistemas computacionales

Compiladores I

Blanca Guadalupe Estrada Renteria



UNIVERSIDAD AUTÓNOMA
DE AGUASCALIENTES

Contenido

DESCRIPCIÓN GENERAL.....	3
ARQUITECTURA DEL SISTEMA	3
FASES DEL COMPILADOR	3
ESPECIFICACION LEXICA.....	4
PALABRAS CLAVE DE SKULD	4
PALABRAS CLAVE ALIAS	5
OPERADORES LÓGICOS.....	6
OPERADORES DE UN CARÁCTER.....	6
OPERADORES DE DOS CARACTERES.....	7
LITERALES	7
COMENTARIOS	8
ERRORES.....	8
GRÁMATICA.....	9
CÓDIGO FUENTE ANALISADOR SINTÁCTICO	11
ARCHIVO CON VISUALIZACIÓN GRÁFICA DEL ÁRBOL SINTÁCTICO.....	11
ARCHIVO CON LISTA DE ERRORES SINTÁCTICOS DETECTADOS	11
ARCHIVOS DE PRUEBA	11
EJEMPLOS ANALISIS SINTÁCTICO.....	12
EJEMPLOS DE ARCHIVOS DE PRUEBA.....	15

DESCRIPCIÓN GENERAL

Skuld es un lenguaje de programación imperativo de alto nivel con sintaxis propia inspirada en la serie Steins;Gate. Permite una escritura mixta: el programador puede utilizar palabras clave exclusivas de Skuld (worldline, gate, choice, loop, etc.) o sus equivalentes en C/TINY clasico (int, main, if, while, etc.), lo que hace el lenguaje accesible.

El compilador esta implementado completamente en Python 3 y sigue el modelo del compilador TINY descrito por Kenneth Loudon en "Compiler Construction: Principles and Practice" (1997). El IDE que lo envuelve fue construido con PyQt5.

ARQUITECTURA DEL SISTEMA

El IDE y el compilador son modulos completamente separados. El IDE invoca al compilador mediante llamadas al sistema (system call) pasando flags de fase:

Léxico	Ejecuta análisis léxico
Sintactico	Ejecuta análisis sintáctico
Semántico	Ejecuta análisis semántico
Intermedio ejecutar	Genera código intermedio
ejecutar	Ejecuta el programa compilado

FASES DEL COMPILADOR

#	FASE	DESCRIPCIÓN
1	Análisis Léxico	Tokenización del código fuente
2	Análisis Sintáctico	Construcción del AST (árbol abstracto)
3	Análisis Semantico	Validación de tipos y semántica
4	Generación de código intermedio	Código de tres direcciones u otra representación
5	Ejecución	Ejecución del programa compilado

ESPECIFICACION LEXICA

PALABRAS CLAVE DE SKULD

Palabra Reservada (Skuld)	Token Asociado	Significado / Equivalente en C/C++
labmem	KW_LABMEM	Modificador de declaración de variable
worldline	KW_WORLDLINE	Tipo entero (equivale a int)
divergence	KW_DIVERGENCE	Tipo flotante/real (equivale a float)
reading	KW_READING	Tipo cadena (equivale a string)
gate	KW_GATE	Bloque principal del programa (equivale a main)
choice	KW_CHOICE	Condicional (equivale a if)
else	KW_ELSE	Alternativa del condicional
loop	KW_LOOP	Bucle con condición al inicio (equivale a while)
pulse	KW_PULSE	Bucle con condición al final (equivale a do)
seal	KW_SEAL	Cierre de bloque sin llaves (equivale a end)
dmail	KW_DMAIL	Salida estándar (equivale a cout / printf)
sphone	KW_SPHONE	Entrada estándar (equivale a cin / scanf)
steiner	KW_STEINER	Definición de función de usuario
void	KW_VOID	Tipo de retorno vacío
fork	KW_FORK	(reservado)
path	KW_PATH	(reservado)
shift	KW_SHIFT	(reservado)

jump	KW_JUMP	(reservado)
return	KW_RETURN	Retorno de valor en función
true	KW_TRUE	Literal booleano verdadero
false	KW_FALSE	Literal booleano falso

PALABRAS CLAVE ALIAS

Alias	Token	Equivalente Skuld / Significado
int	KW_INT	worldline
float	KW_FLOAT	divergence
real	KW_FLOAT	divergence
string	KW_STRING	reading
bool	KW_BOOL	<i>(tipo booleano)</i>
if	KW_IF	choice
then	KW_THEN	<i>(inicio de bloque sin llaves, estilo TINY)</i>
while	KW_WHILE	loop
do	KW_DO	pulse
end	KW_END	seal
until	KW_UNTIL	<i>(condición de terminación en do-until)</i>
main	KW_MAIN	gate
cin	KW_CIN	sphone
cout	KW_COUT	dmail
read	KW_READ	<i>(reservado)</i>
write	KW_WRITE	<i>(reservado)</i>
switch	KW_SWITCH	<i>(reservado)</i>
case	KW_CASE	<i>(reservado)</i>

OPERADORES LÓGICOS

Lexema	Token
and	AND_OP
or	OR_OP
not	NOT_OP

OPERADORES DE UN CARÁCTER

Carácter	Token
=	ASSIGN
+	PLUS
-	MINUS
*	TIMES
/	DIV
%	MOD
<	LT
>	GT
!	NOT_OP
(	LPAREN
)	RPAREN
{	LBRACE
}	RBRACE
[	LBRACKET
]	RBRACKET
;	SEMICOLON
,	COMMA
.	DOT
:	COLON

OPERADORES DE DOS CARACTERES

Lexema	Token
==	EQ
!=	NEQ
<=	LTE
>=	GTE
++	INC
--	DEC
+=	PLUS_ASSIGN
-=	MINUS_ASSIGN
*=	TIMES_ASSIGN
/=	DIV_ASSIGN
%=	MOD_ASSIGN
&&	AND_OP
	OR_OP

LITERALES

Tipo	Token	Ejemplo
Entero	INTEGER_LITERAL	42, 0, 1024
Real/Flotante	FLOAT_LITERAL	3.14, 1.0e-3, 24.0
Cadena	STRING_LITERAL	"Hola mundo", "El Psy Kongroo"
Identificador	IDENTIFIER	x, suma, worldline_var

COMENTARIOS

Tipo	Sintaxis	Descripción
Línea simple	<> texto	Del token <> hasta fin de línea
Bloque	<! texto !>	Multilínea, delimitado por <! y !>

ERRORES

Elemento	Detalle / Especificación
Errores Detectados	• Carácter no reconocido • Cadena no terminada (sin comilla de cierre) • Secuencia de escape inválida dentro de una cadena • Número mal formado (cero a la izquierda, punto sin dígitos, sufijo alfa) • Comentario de bloque no terminado
Formato del Error	ERROR_LEXICO(línea, columna): descripción -> 'lexema'

GRÁMATICA

programa → { definicion_funcion | declaracion_variable } bloque_principal

bloque_principal → gate { lista_sentencias } | main () { lista_sentencias }

lista_sentencias → lista_sentencias sentencia | lista_sentencias
declaracion_variable | ε

declaracion_variable → [labmem] tipo identificador [= expresion] { ,
identificador [= expresion] } ;

identificador → id | identificador , id

tipo → worldline | divergence | reading | int | float | real | bool | string

definicion_funcion → steiner tipo_retorno id ([lista_parametros]) {
lista_sentencias }

tipo_retorno → tipo | void

lista_parametros → lista_parametros , tipo id | tipo id

sentencia → seleccion | iteracion | repeticion | sent_in | sent_out | asignacion |
retorno | llamada_funcion | ;

asignacion → id = expresion ; | id += expresion ; | id -= expresion ; | id *=
expresion ; | id /= expresion ; | id %= expresion ; | id ++ ; | id -- ;

seleccion → choice (expresion) { lista_sentencias } [else { lista_sentencias }] | if
expresion then lista_sentencias [else lista_sentencias] end | if expresion then
lista_sentencias [else lista_sentencias] seal

iteracion → loop (expresion) { lista_sentencias } | while expresion do
lista_sentencias end | while expresion do lista_sentencias seal

repeticion → pulse { lista_sentencias } while (expresion) ; | do lista_sentencias
while (expresion) ; | do lista_sentencias until expresion ;

sent_in → sphone (id) ; | cin >> id ;

sent_out → dmail (salida { , salida }) ; | cout << salida { << salida } ;

salida → expresion | cadena | cadena << expresion | expresion << cadena

retorno → return [expresion] ;

llamada_funcion → id ([lista_args]) ;

lista_args → lista_args , expresion | expresion

expresion → expresion_simple [rel_op expresion_simple]

rel_op → < | <= | > | >= | == | !=

expresion_simple → expresion_simple suma_op termino | termino

suma_op → + | - | or | ||

termino → termino mult_op factor | factor

mult_op → * | / | % | and | &&

factor → factor ^ componente | componente

componente → (expresion) | numero | id | id ([lista_args]) | bool | cadena |
op_unario componente

op_unario → ! | - | +

bool → true | false

numero → entero | real

entero → digito { digito } *(sin cero a la izquierda, salvo 0)*

real → entero . digito { digito } [exponente]

exponente → (e | E) [+ | -] digito { digito }

cadena → " cualquier_texto " *(sin saltos de línea literales)*

CÓDIGO FUENTE ANALISADOR SINTÁCTICO

Le paso la carpeta completa también ahí están todos los archivos que le comparto individualmente

<https://drive.google.com/drive/folders/1DEfDk3JY06YKT5wB8xw4TJvXUhnKB4oP?usp=sharing>

Aquí el código fuente individual:

<https://drive.google.com/file/d/1TTc9ZSYsGJyrtB0AMslEPFXJyMK79LCG/view?usp=sharing>

ARCHIVO CON VISUALIZACIÓN GRÁFICA DEL ÁRBOL SINTÁCTICO

(Este en base al testlexico.txt que nos dio para el análisis léxico, pero modificado para su correcto funcionamiento)

https://drive.google.com/file/d/134U_DzeChmeW9yo7Bo2i3HtsHZ9Xn2Bi/view?usp=sharing

ARCHIVO CON LISTA DE ERRORES SINTÁCTICOS DETECTADOS

Este es en base al ejemplo que se llama **ejemplo_sintactico_con_errores.stn**

<https://drive.google.com/file/d/1fpIN-Ycy27BdVL-mfEQ44kD0f9Oj9x4G/view?usp=sharing>

ARCHIVOS DE PRUEBA

validos

1. <https://drive.google.com/file/d/1h2pdRNsaY8MJfYJ3FixF6NEZjkD853x7/view?usp=sharing>
2. <https://drive.google.com/file/d/1R6ZyWuczIzej6bdE9pmEQXQ4cFri6aZc/view?usp=sharing>

No valido

1. <https://drive.google.com/file/d/1fpIN-Ycy27BdVL-mfEQ44kD0f9Oj9x4G/view?usp=sharing>

EJEMPLOS ANALISIS SINTÁCTICO

Ahora haremos algunos ejemplos para mostrar el funcionamiento

```
<> El Psy Kongroo
gate {
    dmail("Hola, mundo!");
}
```

Analizadores

- Tokens
- Sintáctico
- Semántico
- Intermedio
- Símbolos

- ▼ [Bloque de Código / Programa]
- ▼ [Bloque de Código / gate]
- ▼ [Salida / dmail (cout)]
- [Cadena] "Hola, mundo!"

```
labmem worldline x, y, z;
labmem divergence pi = 3.14;

gate {
    x = 10;
    y = 20;
    z = x + y;
    dmail(z);
}
```

Analizadores

- Tokens
- Sintáctico
- Semántico
- Intermedio
- Símbolos

- ▼ [Bloque de Código / Programa]
- [Declaración de Variable] Tipo: worldline, ID: x
- [Declaración de Variable] Tipo: worldline, ID: y
- [Declaración de Variable] Tipo: worldline, ID: z
- ▼ [Declaración de Variable] Tipo: divergence, ID: pi (Iniciada)
- [Constante] Valor: 3.14
- ▼ [Bloque de Código / gate]
- ▼ [Asignación] ID: x
- [Constante] Valor: 10
- ▼ [Asignación] ID: y
- [Constante] Valor: 20
- ▼ [Asignación] ID: z
- ▼ [Operador] '+'
- [Variable / ID] Nombre: x
- [Variable / ID] Nombre: y
- ▼ [Salida / dmail (cout)]
- [Variable / ID] Nombre: z

```

gate {
  labmem worldline n = 5;
  choice (n > 0) {
    dmail("Positivo");
  } else {
    dmail("No positivo");
  }
}

```

<! Equivalente con sintaxis combinada>

```

gate {
  labmem worldline n = 5;
  if n > 0 then
    dmail("Positivo");
  else
    dmail("No positivo");
  end
}

```

```

1 gate {
2   labmem worldline n = 5;
3   choice (n > 0) {
4     dmail("Positivo");
5   } else {
6     dmail("No positivo");
7   }
8 }
9

```

<! Equivalente con sintaxis combinada!>

```

1 gate {
2   labmem worldline n = 5;
3   if n > 0 then
4     dmail("Positivo");
5   else
6     dmail("No positivo");
7   end
8 }
9

```

Tokens	Sintáctico	Semántico	Intermedio	Simbolos
		✓ [Declaración de Variable] Tipo: worldline, ID: n (Iniciada) [Constante] Valor: 5 ✓ [Condicional / choice (if)] ✓ [Bloque de Código / Condición] ✓ [Operador] '>' [Variable / ID] Nombre: n [Constante] Valor: 0 ✓ [Bloque de Código / Rama Entonces] ✓ [Salida / dmail (cout)] [Cadena] "Positivo" ✓ [Bloque de Código / Rama Sino] ✓ [Salida / dmail (cout)] [Cadena] "No positivo"		
		✓ [Bloque de Código / gate] ✓ [Declaración de Variable] Tipo: worldline, ID: n (Iniciada) [Constante] Valor: 5 ✓ [Condicional / choice (if)] ✓ [Bloque de Código / Condición] ✓ [Operador] '>' [Variable / ID] Nombre: n [Constante] Valor: 0 ✓ [Bloque de Código / Rama Entonces] ✓ [Salida / dmail (cout)] [Cadena] "Positivo" ✓ [Bloque de Código / Rama Sino] ✓ [Salida / dmail (cout)] [Cadena] "No positivo"		

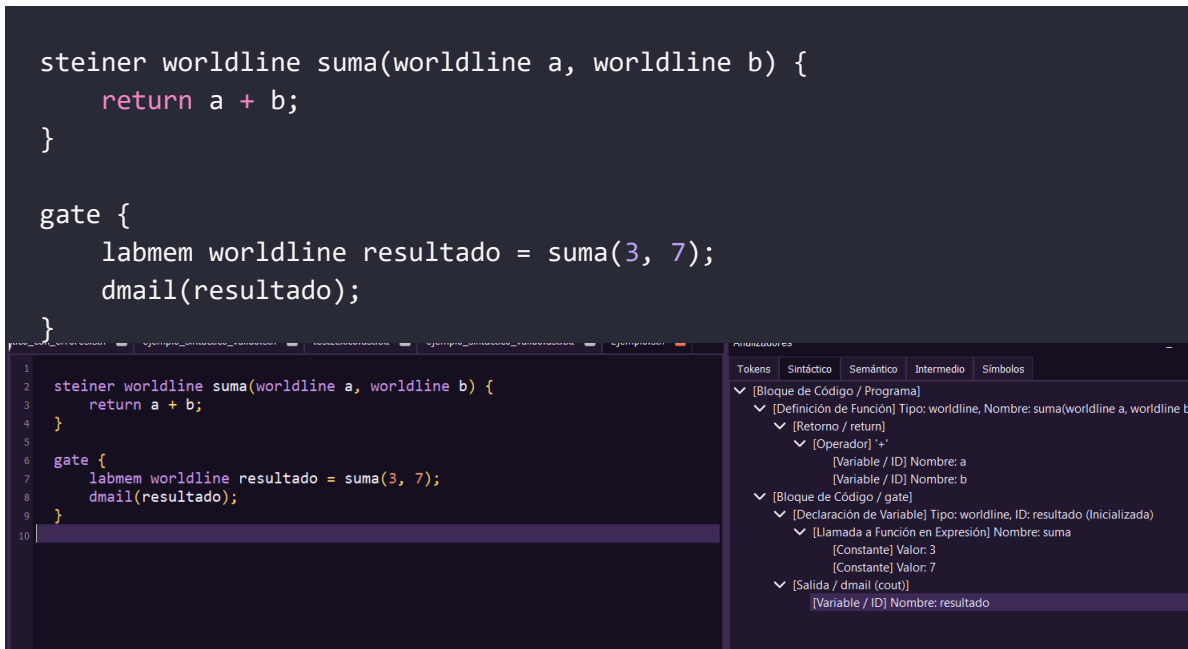
```
gate {
    labmem worldline i = 0;
    loop (i < 10) {
        dmail(i);
        i += 1;
    }
}
```

Tokens	Sintáctico	Semántico	Intermedio	Símbolos
[Bloque de Código / Programa]				
[Bloque de Código / gate]				
[Declaración de Variable] Tipo: worldline, ID: i (Iniciada)				
[Constante] Valor: 0				
[Bucle / loop (while)]				
[Bloque de Código / Condición]				
[Operador] '<'				
[Variable / ID] Nombre: i				
[Constante] Valor: 10				
[Bloque de Código / Cuerpo del Bucle]				
[Salida / dmail (cout)]				
[Variable / ID] Nombre: i				
[Asignación] ID: i				
[Operador] '+'				
[Variable / ID] Nombre: i				
[Constante] Valor: 1				

```
gate {
    labmem worldline i = 0;
    pulse {
        dmail(i);
        i++;
    } while (i < 5);
}
```

```
1 <>Do while ejemplo
2 gate {
3     labmem worldline i = 0;
4     pulse {
5         dmail(i);
6         i++;
7     } while (i < 5);
8 }
9
```

Tokens	Sintáctico	Semántico	Intermedio	Símbolos
[Bloque de Código / Programa]				
[Bloque de Código / gate]				
[Declaración de Variable] Tipo: worldline, ID: i (Iniciada)				
[Constante] Valor: 0				
[Bucle / pulse (do-while)]				
[Bloque de Código / Cuerpo del Bucle]				
[Salida / dmail (cout)]				
[Variable / ID] Nombre: i				
[Asignación] ID: i				
[Operador] '+'				
[Variable / ID] Nombre: i				
[Constante] Valor: 1				
[Bloque de Código / Condición]				
[Operador] '<'				
[Variable / ID] Nombre: i				
[Constante] Valor: 5				



EJEMPLOS DE ARCHIVOS DE PRUEBA

<> Archivo de prueba Skuld

```
labmem worldline x = ; <> Error 1: Falta la expresión de inicialización  
labmem divergence ratio = 1.048596;
```

```

gate {
    x = 10
    dmail("Hola");    <> Error 2: Falta el punto y coma ';' en la
asignación de x

    choice (ratio > 1.0) {
        dmail("Divergencia correcta");
    else {              <> Error 3: Falta cerrar la llave '}' de la rama
choice (then) antes del else
        dmail("Divergencia incorrecta");
    }
}

```

```

ejemplo_sintactico_con_errores.stn | ejemplo_sintactico_valido.stn | testLexico.ast.bt | ejemplo_sintactico_valido.ast.bt | Ejempl
1 <> Archivo de prueba Skuld
2
3 labmem worldline x = ; <> Error 1: Falta la expresión de inicialización
4 labmem divergence ratio = 1.048596;
5
6 gate {
7     x = 10
8     dmail("Hola");    <> Error 2: Falta el punto y coma ';' en la asignación
9
10    choice (ratio > 1.0) {
11        dmail("Divergencia correcta");
12    else {              <> Error 3: Falta cerrar la llave '}' de la rama choic
13        dmail("Divergencia incorrecta");
14    }
15 }
16

```

Analizadores

Tokens	Sintáctico	Semántico	Intermedio	Símbolos
ERROR_SINTACTICO(3, 22): Componente de expresión inválido -> ':'				
ERROR_SINTACTICO(8, 5): Se esperaba SEMICOLON, pero se encontró 'KW_DMAIL' -> 'dmail'				
ERROR_SINTACTICO(12, 5): Sentencia o expresión no reconocida -> 'else'				

<> Archivo de prueba Skuld (Analizador Sintáctico Válido)

```
labmem worldline x = 5, y = 15;
labmem divergence ratio = 1.048596;
labmem reading es_activo = true;

steiner worldline duplicar(worldline valor) {
    labmem worldline resultado = valor * 2;
    return resultado;
}

gate {
    dmail("--- Calculando Divergencia ---");
    x = duplicar(x);

    choice (x < y) {
        dmail("Valor duplicado es menor que y.");
        dmail("x = " + x);
    } else {
        dmail("Valor duplicado es mayor o igual que y.");
        dmail("x = " + x);
    }

    loop (x > 0) {
        dmail("Decrementando x: " + x);
        x = x - 1;
    }

    do {
        dmail("Bucle pulse ejecutado una vez");
    } while (x != 0);
}
```

The screenshot displays the Skuld (Analizador Sintáctico Válido) interface. The left pane shows the source code, and the right pane shows the AST analysis.

Source Code (Left Pane):

```
1 <> Archivo de prueba Skuld (Analizador Sintáctico Válido)
2
3 labmem worldline x = 5, y = 15;
4 labmem divergence ratio = 1.048596;
5 labmem reading es_activo = true;
6
7 steiner worldline duplicar(worldline valor) {
8     labmem worldline resultado = valor * 2;
9     return resultado;
10 }
11
12 gate {
13     dmail("--- Calculando Divergencia ---");
14     x = duplicar(x);
15
16     choice (x < y) {
17         dmail("Valor duplicado es menor que y.");
18         dmail("x = " + x);
19     } else {
20         dmail("Valor duplicado es mayor o igual que y.");
21         dmail("x = " + x);
22     }
23
24     loop (x > 0) {
25         dmail("Decrementando x: " + x);
26         x = x - 1;
27     }
28 }
```

AST Analysis (Right Pane):

The right pane shows the AST analysis, organized into a tree structure. The root node is "[Bloque de Código / Programa]". The tree structure is as follows:

- [Bloque de Código / Programa]
 - [Declaración de Variable] Tipo: worldline, ID: x (Iniciada)
 - [Constante] Valor: 5
 - [Declaración de Variable] Tipo: worldline, ID: y (Iniciada)
 - [Constante] Valor: 15
 - [Declaración de Variable] Tipo: divergence, ID: ratio (Iniciada)
 - [Constante] Valor: 1.048596
 - [Declaración de Variable] Tipo: reading, ID: es_activo (Iniciada)
 - [Constante] Valor: True
 - [Definición de Función] Tipo: worldline, Nombre: duplicar(worldline valor)
 - [Declaración de Variable] Tipo: worldline, ID: resultado (Iniciada)
 - [Operador] "*"
 - [Variable / ID] Nombre: valor
 - [Constante] Valor: 2
 - [Retorno / return]
 - [Variable / ID] Nombre: resultado
 - [Bloque de Código / gate]
 - [Salida / dmail (cout)]
 - [Cadena] "--- Calculando Divergencia ---"
 - [Asignación] ID: x
 - [Llamada a Función en Expresión] Nombre: duplicar
 - [Variable / ID] Nombre: x
 - [Condición / choice (if)]
 - [Bloque de Código / Condición]
 - [Operador] "<"
 - [Variable / ID] Nombre: x
 - [Variable / ID] Nombre: y

```

<> Archivo de prueba
<> El Psy Kongroo!

<> main sum@r 3.14+main)if{32.algo
<> 34.34.34.34

gate {
    labmem worldline x, y, z;
    labmem divergence a, b, c;
    labmem worldline suma = 45;
    labmem worldline mas;

    x = 32;
    x = 23;
    y = 2 + 3 - 1;
    z = y + 7;
    y = y + 1;
    a = 24.0 + 4.0 - 1.0 / 3.0 * 2.0 + 34.0 - 1.0;
    x = (5 - 3) * (8 / 2);
    y = 5 + 3 - 2 * 4 / 7 - 9;
    z = 8 / 2 + 15 * 4;
    y = 14;

    choice (2 > 3) then
        y = a + 3;
    else
        choice (4 > 2) then
            b = 3.2;
        else
            b = 5.0;
        end;
        y = y + 1;
    end;

    a++;
    c--;
    x = 3 + 4;

    do
        y = (y + 1) * 2 + 1;
        loop (x > 7) {
            x = 6 + 8 / 9 * 8 / 3;
            sphone(x);
            mas = 36 / 7;
        };
};

```

```

until (y == 5);

loop (y == 0) {
    sphone(mas);
    dmail(x);
}
}

```

testLexico.bt
ejemplo_sintactico_con_errores.stn
ejemplo_sintactico_valido.stn
testLexico.asf.bt
ejemplo_sintactico_valido.stn

1 <> Archivo de prueba
2 <> El Psy Kongrool!
3
4 <> main sum@r 3.14+main)if{32.algo
5 <> 34.34.34.34
6
7 gate {
8 labmem worldline x, y, z;
9 labmem divergence a, b, c;
10 labmem worldline suma = 45;
11 labmem worldline mas;
12
13 x = 32;
14 x = 23;
15 y = 2 + 3 - 1;
16 z = y + 7;
17 y = y + 1;
18 a = 24.0 + 4.0 - 1.0 / 3.0 * 2.0 + 34.0 - 1.0;
19 x = (5 - 3) * (8 / 2);
20 y = 5 + 3 - 2 * 4 / 7 - 9;
21 z = 8 / 2 + 15 * 4;
22 y = 14;
23
24 choice (2 > 3) then
25 y = a + 3;
26 else
27 choice (4 > 2) then

Analizadores
Tokens
Sintáctico
Semántico
Intermedio
Símbolos

[Bloque de Código / Programa]
[Bloque de Código / gate]
[Declaración de Variable] Tipo: worldline, ID: x
[Declaración de Variable] Tipo: worldline, ID: y
[Declaración de Variable] Tipo: worldline, ID: z
[Declaración de Variable] Tipo: divergence, ID: a
[Declaración de Variable] Tipo: divergence, ID: b
[Declaración de Variable] Tipo: divergence, ID: c
[Declaración de Variable] Tipo: worldline, ID: suma (Iniciada)
[Constante] Valor: 45
[Declaración de Variable] Tipo: worldline, ID: mas
[Asignación] ID: x
[Constante] Valor: 32
[Asignación] ID: x
[Constante] Valor: 23
[Asignación] ID: y
[Operador] '-'
[Operador] '+'
[Constante] Valor: 2
[Constante] Valor: 3
[Constante] Valor: 1
[Asignación] ID: z
[Operador] '+'
[Variable / ID] Nombre: y
[Constante] Valor: 7
[Asignación] ID: y
[Operador] '+'